Ders 2 Çalışma Özeti

Ders 2 Çalışma Özeti

Genel Konular

- Tasarım kalıbı nedir? (ekrandan, 00:11:10)
  - Yazılım tasarımında karşılaşılan belirli sorunların yalın ve güzel çözümlerinin tarifidir. Çözüm somut bir gerçekleme olmak zorunda değildir, soyut bir tarif de olabilir.
  - Kalıplar klasik olarak nesneye yönelim üzerinden önerilmiştir (Dörtlü Çete), ama başka programlama yaklaşımları ve mimari stiller için de kalıplar önerilebilir.
  - Amaç: tekerleği yeniden keşfetmeden "iyi" tasarıma ulaşmak; sorunları esnek biçimde ve yeniden kullanılabilirliği artıracak yönde çözmek. Meslekte tek değişmeyen şey değişimin kendisidir; kalıplar zorunlu değişiklikleri en az çabayla ve tasarımın genel kalitesini bozmadan yapabilmeyi sağlar.
- Bir tasarım kalıbının dört ana bileşeni (ekrandan, 00:11:10)
  - İsim: kalıbı en güzel şekilde anlatan, bir ya da birkaç kelimelik ad. İyi ad, kalıba aşina olunduktan sonra yalnız adından her şeyi hatırlatmalıdır.
  - Sorun: kalıbın çözdüğü sorunun tarifi; sorunun neden zor olduğuna da değinilir.
  - Çözüm: önerilen tasarımın bileşenleri (sınıflar, nesneler, alanlar, metotlar), ilişkileri, sorumlulukları, birlikte çalışmaları; UML sınıf şemalarıyla belgelenir.
  - Tartışma/Sonuçlar: çözümün esneklik, genişletilebilirlik, taşınabilirlik gibi iç kalite özelliklerine etkisi; güçlü ve zayıf yönleri; yer-zaman çelişkileri (time-memory trade-off); başka kalıplarla işbirliği olanakları.
- Kalıplar arası ilişkiler (ekrandan, 00:16:50)
  - Dörtlü Çete'nin kitabındaki şema kalıpların birlikte çalışma olasılıklarını gösterir; örneğin Abstract Factory tek örnek (single instance) olarak Singleton ile, Factory Method ile gerçeklenebilir (implement using), Prototype ile dinamik olarak yapılandırılabilir.
  - Şemayı ezberlemek gerekmez; çalışırken yanda durması, bir kalıp seçildikten sonra problemin geri kalanını modellemede yol gösterir.
- Tasarım kalıplarının yararları (ekrandan, 00:20:00)
  - Tekerleği yeniden icat etmemek, yani zaman kazanmak.
  - Üzerinde birçok deneyimli kişinin emeği geçtiğinden "kaliteli" çözüm.
  - Deneyimin yazılı biçimde aktarılabilmesi.
  - Yazılım ekibi arasında ortak bir sözlük oluşturması: iki-üç kalıp adıyla sistemin önemli bir kısmının nasıl tasarlanacağı anlatılabilir.
- Kalıpların sınıflandırılması (ekrandan, 00:22:45 ve 00:30:00)
  - Amaçlarına göre üç küme: Creational (yaratımsal) nesnelerin oluşturulması, temsili ve ilişkilendirilmesindeki bağımlılığı azaltır; Structural (yapısal) sınıf ve nesneleri bir araya getirerek daha büyük yapılar elde eder; Behavioral (davranışsal) sorumlulukların nesnelere dağıtılması ve algoritma seçimine yöneliktir.
  - Odağına göre iki grup: sınıfsal kalıplar (sınıflar ve alt sınıfları arasındaki, derleme zamanında belirlenen, daha durağan ilişkiler) ve nesnesel kalıplar (çalışma sırasında rahatlıkla değiştirilebilen, daha dinamik nesne ilişkileri). Adapter hem sınıfsal hem nesnesel sayılır; diğer kalıplar yalnız birine girer.
  - Dörtlü Çete'nin ayrıca yedi gruplu bir sınıflaması vardır (ekrandan, 00:32:35): sınıf belirtmeden nesne oluşturabilmek; nesne gerçeklemesine diğer nesnelerin bağımlılığını azaltmak; algoritmik bağımlılığı azaltmak; gevşek ilişkilendirme; hizmet kalıpları; kalıtım yerine parça/bütün ilişkisini öneren kalıplar; sınıf kodlarını daha rahat değiştirmeye yönelik kalıplar. Derste bu sıra değil, kitaptaki üç kümeli sıra izlenir.

- MVC (Model-View-Controller) kalıbı (ekrandan, 00:36:20 ve 01:00:00)
  - Sorun: aynı veriyi farklı biçimlerde göstermek ve kullanıcıya verinin nasıl ve ne kadarının gösterileceğini seçme olanağı vermek.
  - Çözüm: veriyi (Model), verinin gösterimini (View) ve gösterimin yönetimini (Controller) ayrı sınıflar olarak ele almak.
  - Temel ilişkiler (ekrandan, 00:50:00): Controller → View, Controller → Model, View → Model. Model ne View'u ne Controller'ı bilir.
  - Yararlar: gösterim, kontrol ve veriyi ayırarak bunları çalışma anında değiştirebilmek, ilgi alanlarını ayırmak (uyumu yükseltir), yeniden kullanımı artırmak.
  - Çeşitlemeler: Model'in değişiklikleri bildirmesi; birden fazla View (iç içe, ayrı/tek görevcikte, örtüşen/ayrık).
- Abstract Factory kalıbı (ekrandan, 01:26:00, 01:38:10, 01:40:00)
  - Amaç: birbiriyle ilişkili ya da aynı aileye ait nesneleri, somut sınıflarını belirtmeden oluşturabilmek.
  - Örnek sorun: farklı pencere yöneticileri (Motif, Presentation Manager; güncel örnek olarak GNOME, KDE) altında çalışan bir uygulama. İstemci "pencere ver, kaydırma çubuğu ver" demeli, hangi aileden somut sınıfın kullanıldığını bilmemelidir.
  - Yapı: soyut AbstractFactory (CreateProductA, CreateProductB), her aile için bir ConcreteFactory, her ürün türü için soyut AbstractProduct ve ailelere göre somut ürünler. İstemci yalnız soyut fabrika ve soyut ürünler üzerinden çalışır.
  - Uygulanabileceği anlar: sistemin ürünlerin nasıl oluşturulduğundan bağımsız olması istendiğinde; sistemin farklı ailelerden birindeki nesnelerle yapılandırılması gerektiğinde; ilişkili ürün ailelerinin birlikte kullanılması gerektiğinde; bir ürün ailesinin yalnız arayüzleri açığa çıkarılarak sınıf kütüphanesi olarak sunulacağı durumlarda.
- Abstract Factory örneği: 3B oyun motoru (ekrandan, 01:50:00, 02:00:00, 02:08:00)
  - Motor hem OpenGL hem DirectX kütüphanesini desteklemeli; her kütüphaneden Renderer ve Shader bileşenleri gerekir.
  - Çözüm 1, switch-case: `LibraryType` enum'u ve her varyasyon için ayrı üye alan; her işlemde kütüphaneye göre dallanma.
  - Çözüm 2, kalıtım: soyut `Shader` ve `Renderer`, bunların OpenGL ve DirectX alt sınıfları; switch-case yalnız kurucuda kalır.
  - Çözüm 3, Abstract Factory: `ComponentFactory` soyut sınıfı (`createRenderer()`, `createShader()`) ve `OpenGLFactory`, `DirectXFactory` alt sınıfları; `ApControl` fabrikayı kurucuda parametre olarak alır, enum'a gerek kalmaz.

Hocanın Özellikle Vurguladığı Kısımlar

- Switch-case gördüğünüzde alarm zili çalmalı
  - Switch-case kullanıyorsanız ya bir kalıtım ilişkisi kurup çok biçimlilik kullanmanız gerekir ya da sorumlulukları yanlış atamışsınızdır.
  - İki zayıf yönü: metodu aşırı uzatır (anlaşılması ve bakımı zorlaşır), yeni bir seçenek eklendiğinde switch'in geçtiği her yere yeni case eklemek gerekir (kopyala-yapıştır ve bakım kâbusu).
  - Switch-case tamamen yasak değildir: sistem ilklenirken hangi fabrikanın oluşturulacağına karar vermek için tek bir yerde kullanılabilir.
- Kalıtım "ağır sıklet" bir ilişkidir
  - Kalıtım ilişkisi durağandır: bir öğrenci nesnesi bir çalışan nesnesine doğrudan dönüşemez; sahiplik ilişkisi ise çalışma anında değiştirilebilir (bir insan arabasını başkasına satabilir).
  - Deneyimli tasarımcılar mümkün oldukça kalıtım yerine parça/bütün ilişkisini tercih eder, ama kalıtımın "cuk oturduğu" (is-a) yerlerde onu kullanmaktan kaçınmazlar.
  - Kalıplardaki ilk (kötü) çözümler genelde kalıtımın yanlış kullanıldığı çözümler olacaktır.
- MVC denince önce tasarım kalıbı düşünülmeli
  - MVC, Microsoft'un MVC framework'ünden çok önce (Dörtlü Çete kitabı 1994) bilinen bir tasarım kalıbıdır; framework bu kalıba dayanır.
- Sınavda ve çizimlerde güncel UML gösterimi
  - Dörtlü Çete kitabındaki şemalar eski gösterimdedir; kendi çizimlerinizde güncel standarda uyun.
  - Elle çizimde italik yazı ayırt edilemeyeceği için soyut sınıflar «abstract» stereotipiyle,

soyut metotlar imzadan sonra `{abstract}` etiketiyle gösterilmelidir.
- İlk uygun kalıpta durmayın
  - Bir problem için kalıp listesini tek tek gözden geçirin; ilk işe yarayan kalıbı bulunca durmayın, daha uygun bir kalıp ya da alt problemleri farklı kalıplarla çözme olanağı çıkabilir.
- Kalıp seçimi bir değerlendirmedir
  - Abstract Factory çözümü üç yeni sınıf getirir; kalıtımlı çözüm de seçilebilir. Hoca, nesne türünü belirleme sorumluluğunu ayrı bir sınıfa verip uyumu artırdığı için kalıplı çözümü tercih eder.

Kısa Tekrar Notları

- Tasarım kalıbı: belirli sorunların yalın ve güzel çözümlerinin tarifi; somut kod şart değil.
- Dört bileşen: isim, sorun, çözüm, tartışma/sonuçlar.
- Yararlar: tekerleği yeniden icat etmemek, kaliteli çözüm, deneyimin yazılı aktarımı, ekip içinde ortak sözlük.
- Üç küme: yaratımsal, yapısal, davranışsal. İki odak: sınıfsal (durağan) ve nesnesel (dinamik); Adapter ikisinde de.
- MVC: Model = veri, View = gösterim, Controller = kullanıcı komutlarını işleyen yönetici. Asgari ilişkiler: C→V, C→M, V→M.
- Model'in anlık değişikliği bildirmesi gerekiyorsa Model→Controller oku eklenir (ileride Observer kalıbı); bu durumda Model→View okuna gerek kalmaz.
- Controller'ın View ile ilişkisinde `1..*`: Controller bir dizi, liste, ağaç ya da map gibi bir veri yapısında birden çok View tutar.
- Abstract Factory: ilişkili nesne ailelerini somut sınıf adı vermeden oluşturur. Adında "Factory" geçen kalıpta nesne üreten bir metot ya da sınıf vardır.
- Switch-case → kalıtım + çok biçimlilik; ya da sorumluluklar yanlış atanmış.
- Tasarım ilkeleri: neyin değiştiğini bul ve onu sarmala; kalıtım yerine parça/bütün ilişkisini tercih et; gerçeklemelere göre değil arayüzlere göre tasarla.
- Kara kutu (black box): yalnız dışa sunulan arayüzle kullanılan bileşen; beyaz kutu (white box): kullanmak için iç yapısını bilmek gerekir.
- Yanlış kalıtım: `ApControl`'dan `OpenGLApControl` ve `DirectXApControl` türetmek kod tekrarına yol açar.

Detaylı Açıklamalar

Tasarım kalıbı kavramı ve kalıpların anlatım sırası

Tasarım kalıbı, yazılım tasarımında sık karşılaşılan belirli sorunların yalın ve güzel çözümlerinin tarifidir. Terim mimarlıktan ödünç alınmıştır; hocanın verdiği mimari örnekte, içinde bir insanın günde bir saatten fazla vakit geçireceği her yerde bir pencere bulunmalıdır. Kalıp önerisinin ayrıntı düzeyi çok değişebilir: derste ilk işlenen MVC kalıbı hiç kaynak kod içermez, ilerideki kalıplar çok daha ayrıntılıdır. Hocaya göre birinin kalıp dediği tavsiyeye bir başkası "temel bir ilke" diyebilir; iki taraf da kendi açısından haklıdır, ama iyi bir fikir her zaman değerlidir.

Kalıbın tartışma bölümü, çözümün esneklik, genişletilebilirlik, taşınabilirlik gibi iç kalite özelliklerine etkisini, güçlü ve zayıf yönlerini, yer-zaman dengesini ve başka kalıplarla işbirliği olanaklarını ele alır. İşbirliğine hocanın verdiği örnek: Strategy kalıbındaki her strateji gerçeklemesi bir Singleton olabilir. Bir ekipte kıdemli bir tasarımcı "şu kısımda Strategy kullanalım, stratejilerde Template Method da iyi olur" dediğinde, iki cümleyle sistemin önemli bir kısmının nasıl tasarlanacağı anlatılmış olur; ortak sözlük yararı budur.

Derste Dörtlü Çete kitabındaki sıra izlenir: önce yaratımsal, sonra yapısal, sonra davranışsal kalıplar (ekrandan, 00:31:05). Hoca daha önce yedi gruplu sınıflamaya göre anlatmış, ama bir kalıbı anlatırken henüz görülmemiş bir kalıba başvurmak gerektiğinde anlatım havada kaldığı için kitabın sırasına dönmüştür. Interpreter kalıbı kitapta 15. sırada anlatılır, derste sona alınmıştır.

MVC kalıbı ve çeşitlemeleri

Hoca kalıbı bir seçim sonucu örneğiyle açıkladı (ekrandan, 00:40:00). Excel'deki tabloda sekiz partinin oy oranları vardı: A %35, B %28, C %17, D %10, E %5, F %2, G %2, H %1. Aynı veri pasta grafik, çubuk grafik ya da tablo olarak gösterilebilir; kullanıcı barajın altında kalan partileri "diğer" diye birleştirmek, il il ya da ittifak bazında göstermek isteyebilir. Bu sorunda üç ayrı bileşen vardır: veri, gösterim biçimi ve gösterimin yönetimi. İyi tasarım tanımı gereği tasarım da bu üç bileşeni ayrı ayrı modellemelidir.

Akılda kalması için ressam benzetmesi kullanılır: resmi yapılan kişi modeldir (Model), resmi nasıl çizeceğine karar veren ve fırçayı vuran ressam kontrolcüdür (Controller), farklı açılardan ya da tarzlarda yapılan resimler de görünümlerdir (View).

Temel ilişkiler en az düzeydedir: Controller hem View'a hem Model'e, View Model'e bağlıdır; Model hiçbirini bilmez, çünkü hangi açıdan bakılırsa bakılsın veri veridir. Gereksinimler bu yapıya ekleme yaptırabilir. Veri yalnız Controller üzerinden değil, örneğin sahadaki sensörlerden de değişiyorsa, Controller'ın modeli belirli aralıklarla yoklaması mümkündür; ama sık yoklama iletişim maliyetini artırır, seyrek yoklama ise değişikliklerden geç haberdar olmak demektir. Anlık bildirim gerekiyorsa Model'den Controller'a bir ilişki çizilir (ileride işlenecek Observer kalıbı bu konudadır); Controller View'ları tazeleyebileceği için Model'den View'a ayrıca ok gerekmez. Birden fazla View olduğunda (BarChart, PieChart, Table ortak bir View üst sınıfından türer) View listesini tutmak Controller'ın işidir. Birden fazla Controller da olabilir (klavye, ses, dokunmatik ekran, tablet kalemi); bu durumda değişikliği View üzerinden yaymak, kardeş sınıfların birbirinden haberdar olmasını gerektireceği için daha zordur.

Abstract Factory ve oyun motoru örneği

Abstract Factory'nin örnek sorununda istemci, pencere yöneticisinden bağımsız olarak "pencere çiz, kaydırma çubuğu çiz" diyebilmelidir. Somut sınıfları bilmek zorunda kalırsa hem karmaşıklık artar hem de bu sınıflardaki her değişiklik istemci kodunu da değiştirir. Çözümde istemci soyut `WidgetFactory` ile ilişkilidir; `MotifWidgetFactory` ve `PMWidgetFactory` bu sınıftan türer ve kendi ailelerinin `Window` ve `ScrollBar` nesnelerini üretir (ekrandan, 01:26:00).

Oyun motoru örneğinde üç çözüm karşılaştırıldı:

- Çözüm 1 (switch-case): `LibraryType` enum'u (`OpenGL`, `DirectX`), ayrı `OpenGLRenderer`, `OpenGLShader`, `DirectXRenderer`, `DirectXShader` sınıfları ve `ApControl` içinde her varyasyon için bir üye alan (ekrandan, 01:50:00). DirectX 10, 11 ve 12'yi ayrı ayrı desteklemek gerektiğinde enum'a seçenek, yeni sınıflar, yeni üye alanlar ve her işlemdeki switch'e yeni case'ler eklemek gerekir.
- Çözüm 2 (kalıtım): "Neyin değiştiğini bul ve onu sarmala" ilkesiyle değişen bileşenler (Renderer, Shader) soyut sınıf yapılır. `ApControl` soyut türler üzerinden çalışır; switch-case yalnız kurucuda kalır (ekrandan, 02:00:00). Yeni DirectX sürümleri yalnız yeni kardeş sınıflar, bir enum değeri ve kurucuda bir case gerektirir; `doRendering()` ve `doShading()` değişmez.
- Çözüm 3 (Abstract Factory): `ComponentFactory` soyut sınıfı `createRenderer()` ve `createShader()` soyut metotlarını tanımlar; `OpenGLFactory` ve `DirectXFactory` kendi ailelerinin nesnelerini döndürür. `ApControl` fabrikayı kurucuda parametre olarak alır (ekrandan, 02:08:00):

```
package dp.abstractFactory.solution3;
public class ApControl {
   private ComponentFactory library;
   private Renderer r;
   private Shader s;
   public ApControl( ComponentFactory library ) {
         this.library = library;
         /* Seçenek A: library nesnesini ilklendirerek kütüphane türünü
          * switch-case ile önceki örnekteki gibi belirle
          * Seçenek B: buradaki gibi kurucu metoda parametre olarak ver.
          * Seçenek C: Farklı tür hedef bilgisayarlar için
          * farklı dağıtımlar yap ve alttaki uygun satırın yorumunu kaldır.
         // library = new OpenGLFactory();
```

```
            // library = new DirectXFactory();
            */
            r = this.library.createRenderer();
            s = this.library.createShader();
    }
    void doRendering ( ) {
        r.renderOpA(); r.renderOpB(); r.renderOpC();
    }
    void doShading ( ) {
        s.shadeOpA(); s.shadeOpB(); s.shadeOpC();
    }
}
```

Hoca iki ve üçüncü çözümü Eclipse'in karşılaştırma aracıyla yan yana gösterdi (ekrandan, 02:11:40): fark, enum alanın yerini `ComponentFactory` alanının alması ve kurucudaki switch-case'in ortadan kalkmasıdır. Kalıtımlı çözümde de switch-case ayrı bir sınıfa ya da metoda taşınabilir, ama kalıpta bu sorumluluğun doğal bir yeri (fabrika sınıfı) vardır. Kalıplı çözüm bağlaşımı biraz artırır, buna karşın uyumu iyileştirir; geçen haftaki veritabanı örneğine benzer, ama kazanım o kadar belirgin değildir.

Kalıtımın yanlış kullanımına örnek olarak `ApControl`'dan `OpenGLApControl` ve `DirectXApControl` türetmek gösterildi (ekrandan, 02:20:00). Bu durumda `doRendering()` ve `doShading()` gibi metotlar kardeş sınıflarda kopyalanır; bir değişiklik gerektiğinde aynı değişiklik her kopyada yapılmak zorunda kalınır.

- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.